Paper Review • arXiv 2611.02660

HeLM: Explicit Language Memory for Memory-Dependent VLA Manipulation

HeLM: Hierarchical Explicit Language Memory for Non-Markovian Manipulation

Gyeonghun Kim, Donghoon Kim, Minsoo Choi, Suyun Lee
Seoul National University • First submitted 2026-11-02

Project page: kimspacehuni.github.io/HeLM

TL;DR

VLM writes a text log, VLA follows: big memory-task gains

HeLM uses a fine-tuned VLM as the high-level policy, with **two-stage "event detection → event-triggered update"** reasoning that maintains task progress as structured text memory (**working memory + episodic context**) for the VLA to execute, solving counting, ordering and cross-task recall tasks that current-frame-only VLAs cannot.

85.7%

Intra subtask success

Best baseline 72.9% (+12.8 pp)

89.2%

Inter subtask success

Keyframe baseline only 16.6%

4.5×

Less GPU memory

8 events: <6 GB vs 28 GB

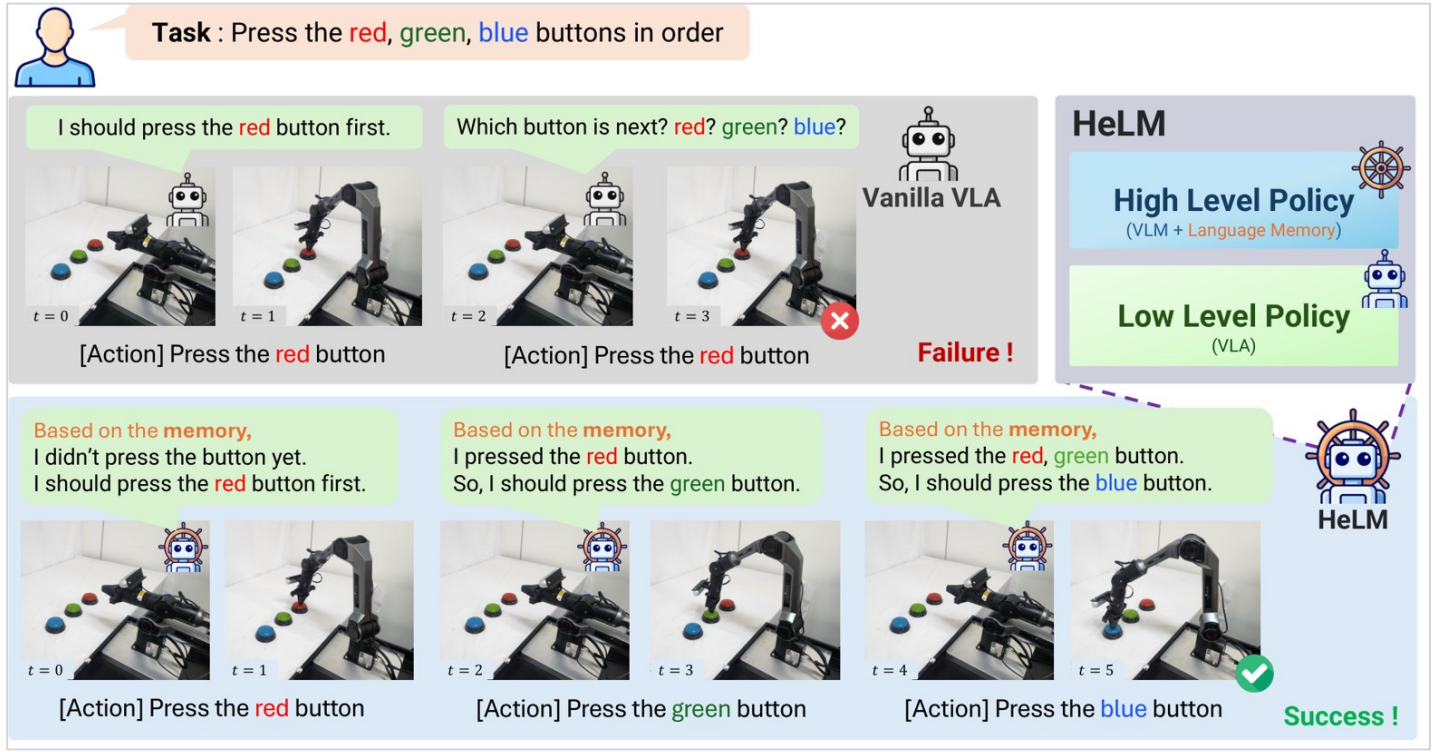
2.5×

Faster long-horizon inference

vs. Keyframe-HLP

Eval.: 10 real-robot non-Markovian tasks (56 variants), AgileX PiPER 7-DoF arm; plus LIBERO-Mem simulation.

Frame-only VLAs forget: same view ≠ same progress



$$P(s_t \mid o_t, l) \neq P(s_t \mid o_{0:t}, l)$$

The current observation is insufficient to infer the true state

Three typical memory needs

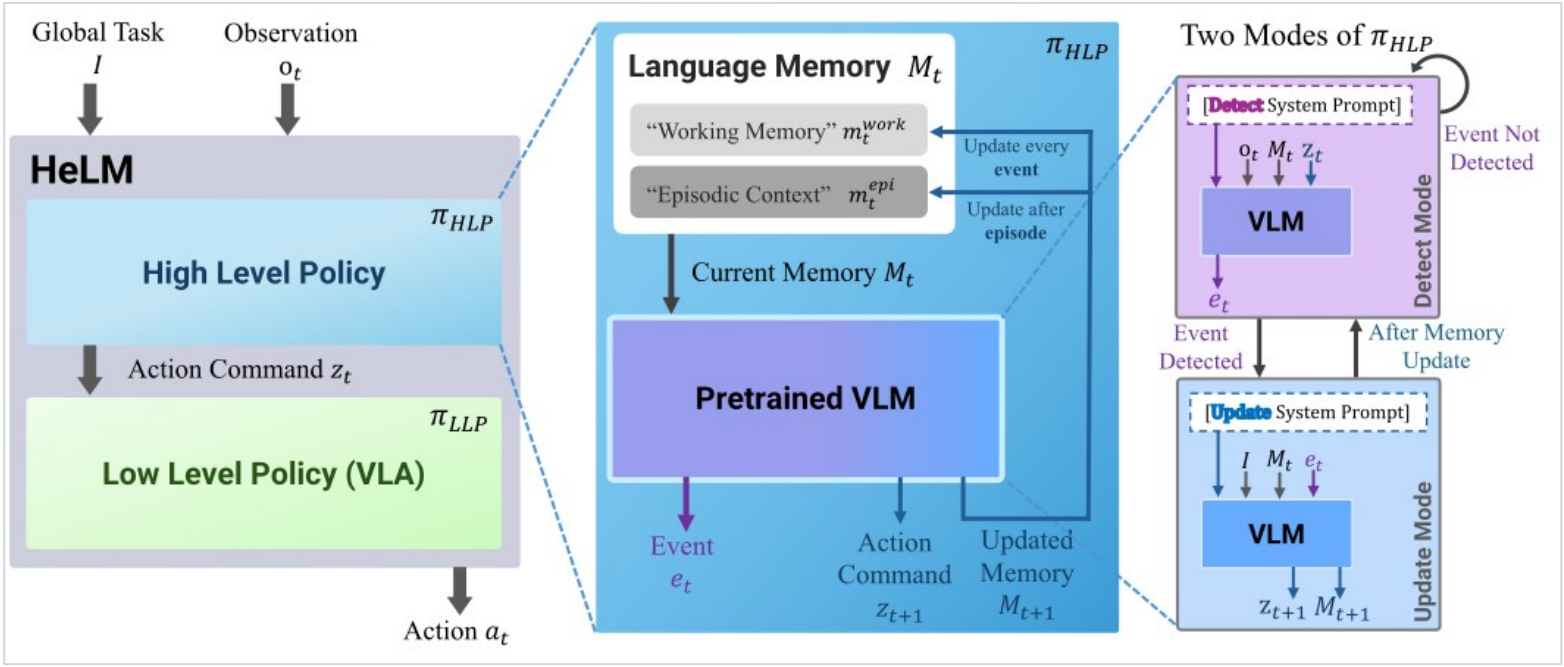
- Counting: how many presses so far?
- Ordering: which steps are done?
- Recall: where was / is the object hidden?

Limits of prior approaches

Latent memory (MemoryVLA, etc.): failures are hard to diagnose

Visual history (multi-frame / keyframe): image tokens accumulate; compute and memory grow linearly over long horizons

Two levels: the VLM "remembers and plans", the VLA "acts"



High-level HLP (π_HLP)

Qwen2.5-VL-7B + QLoRA (r=16)

Maintains language memory M_t, outputs language command z_t

Low-level LLP (π_LLP)

π0 (real robot) / π0.5 (LIBERO-Mem)

z_t + o_t → action chunk, H = 10

Key reformulation

$\Pi(a_t \mid o_t, l) \rightarrow \Pi(a_t \mid o_t, l, M_t)$

Memory tracks task state; the image handles spatial grounding

Two memories + two-stage reasoning: memory changes only on events

Working memory m^work

Within-episode progress, updated after each event

E.g.: Progress: blue (Goal: observe)

Episodic context m^epi

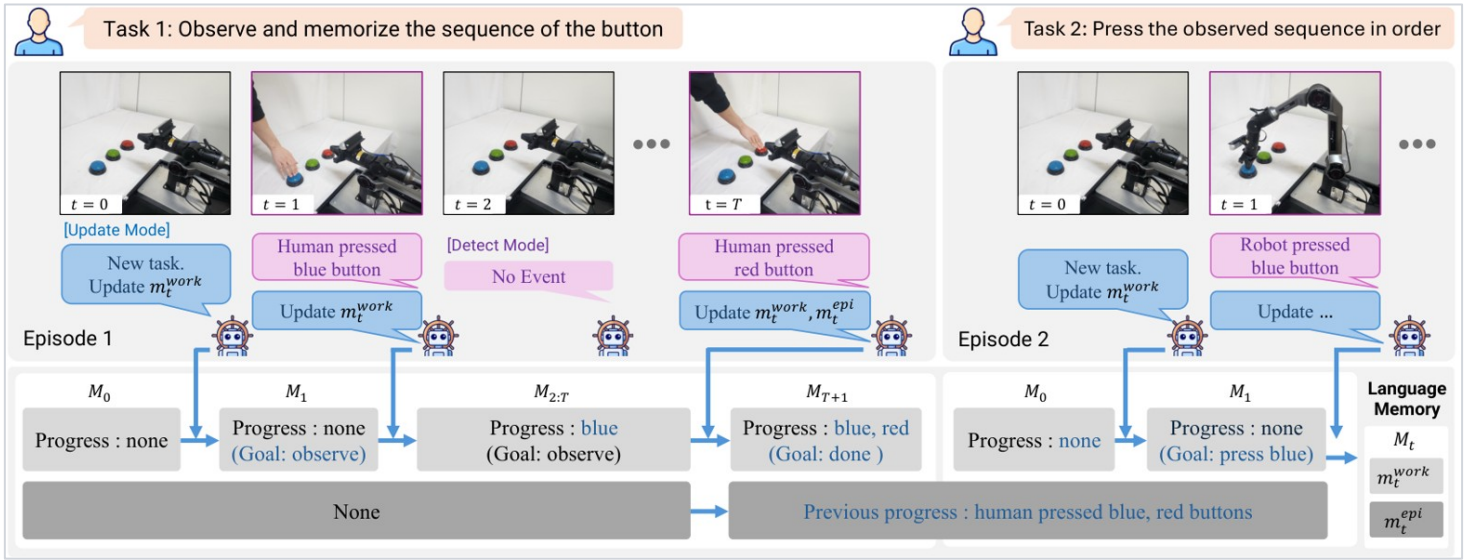
Distilled from working memory at instruction switches; kept across episodes

E.g.: Previous progress: human pressed blue, red

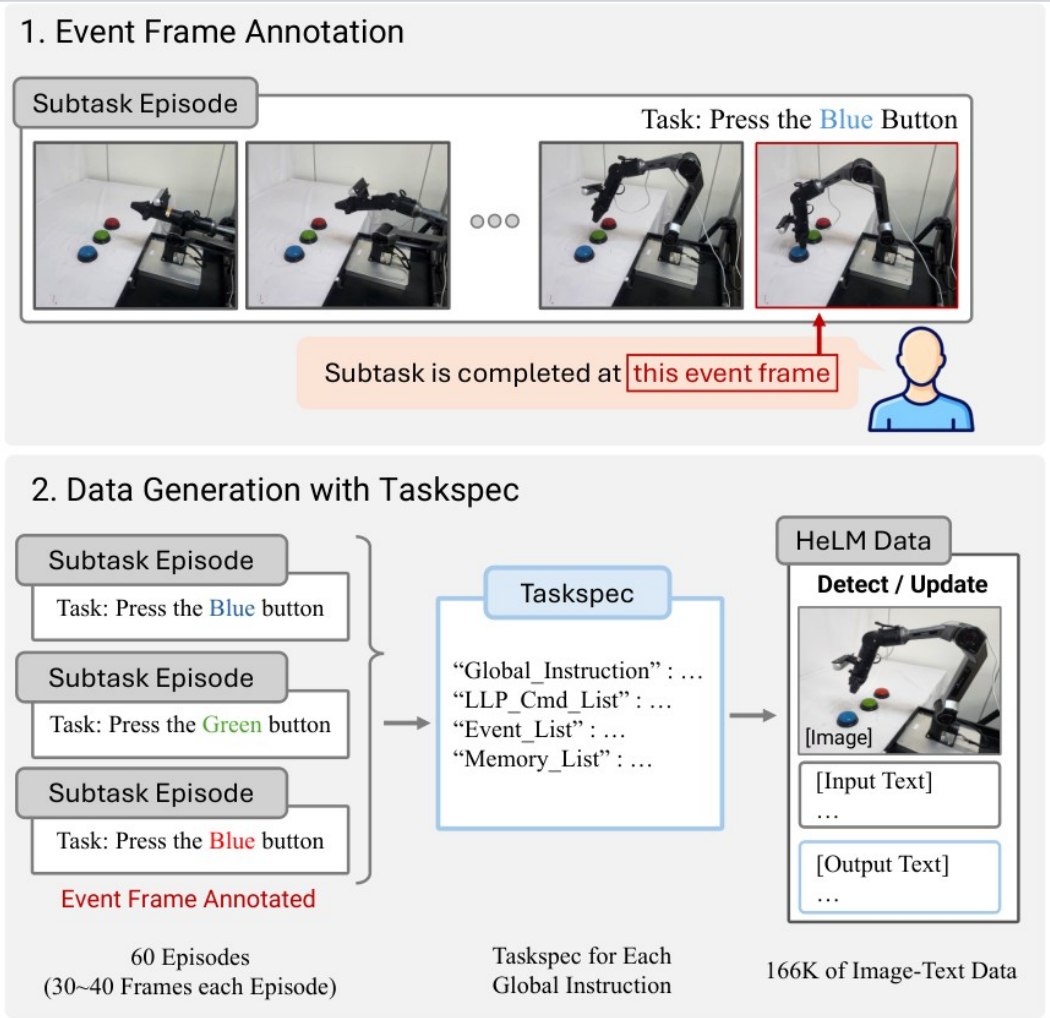
① **Detect:** $e_t \sim \pi_det(o_t, M_t, z_t)$, called at each action-chunk boundary

② **Update:** only if $e_t \neq \varnothing$ or $l_t \neq l_{t-1}$, $(M_{t+1}, z_{t+1}) = \pi_upd(l_t, M_t, e_t)$

Both modes share VLM weights; only the system prompt differs



TaskSpec: logically stitch short clips into long-horizon training data



1 Annotate event frames

Mark "done" frames in short subtask clips, ~5 min / subtask

2 Write TaskSpec

Command & event vocabularies, per-step memory states; GPT can generate variants from templates

3 Synth. Detect/Update

Image + memory text → event / new memory + command

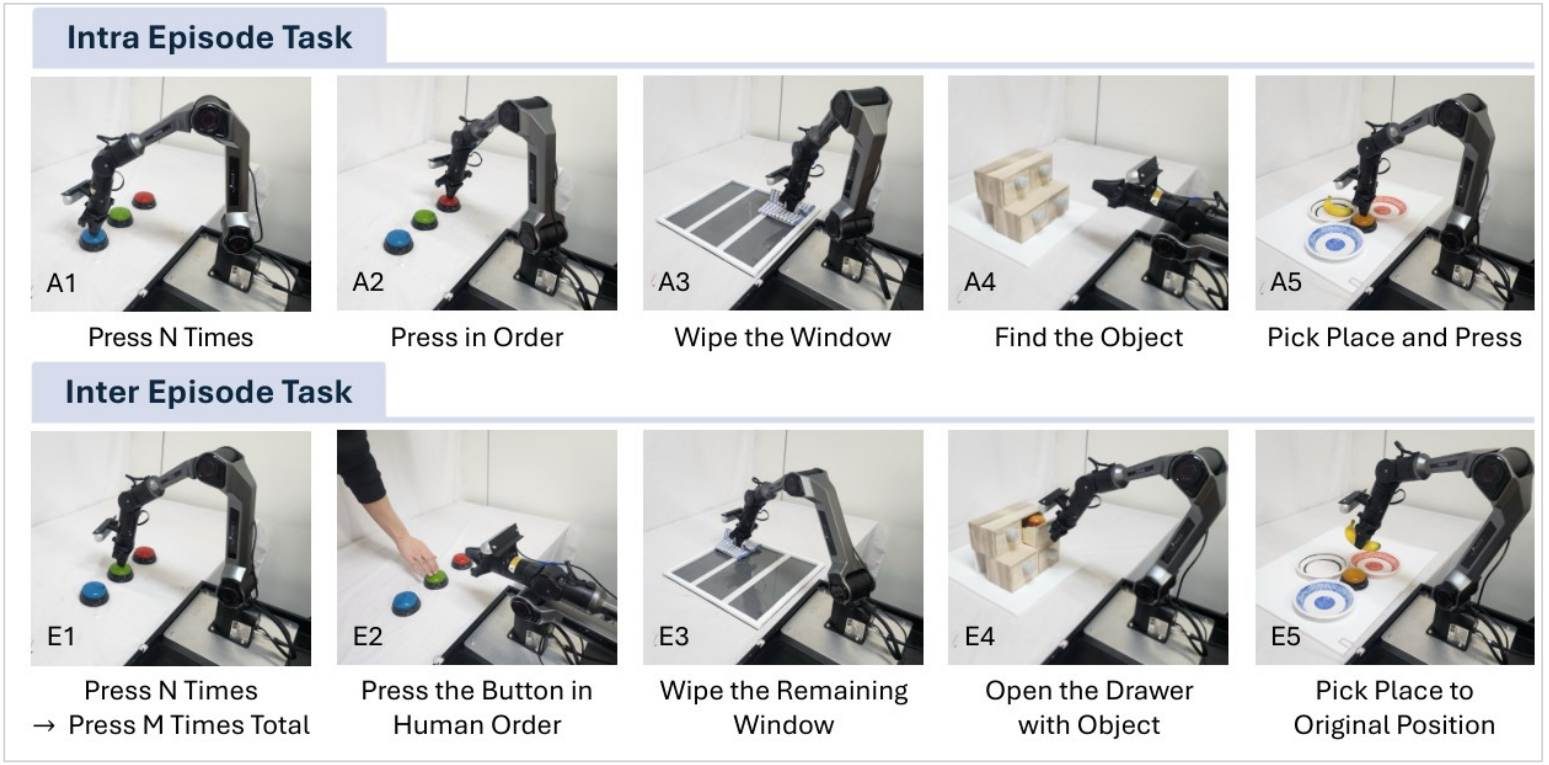
166K	~15 h
Image-text samples	Single A6000

130k train / 36k val; QLoRA 4-bit, 3000 steps

Source: Fig.7, Tab.5, Appendix A

Task Suite

10 real-robot tasks, 56 variants, plus a new inter-episode memory axis



10

Tasks

56

Variants

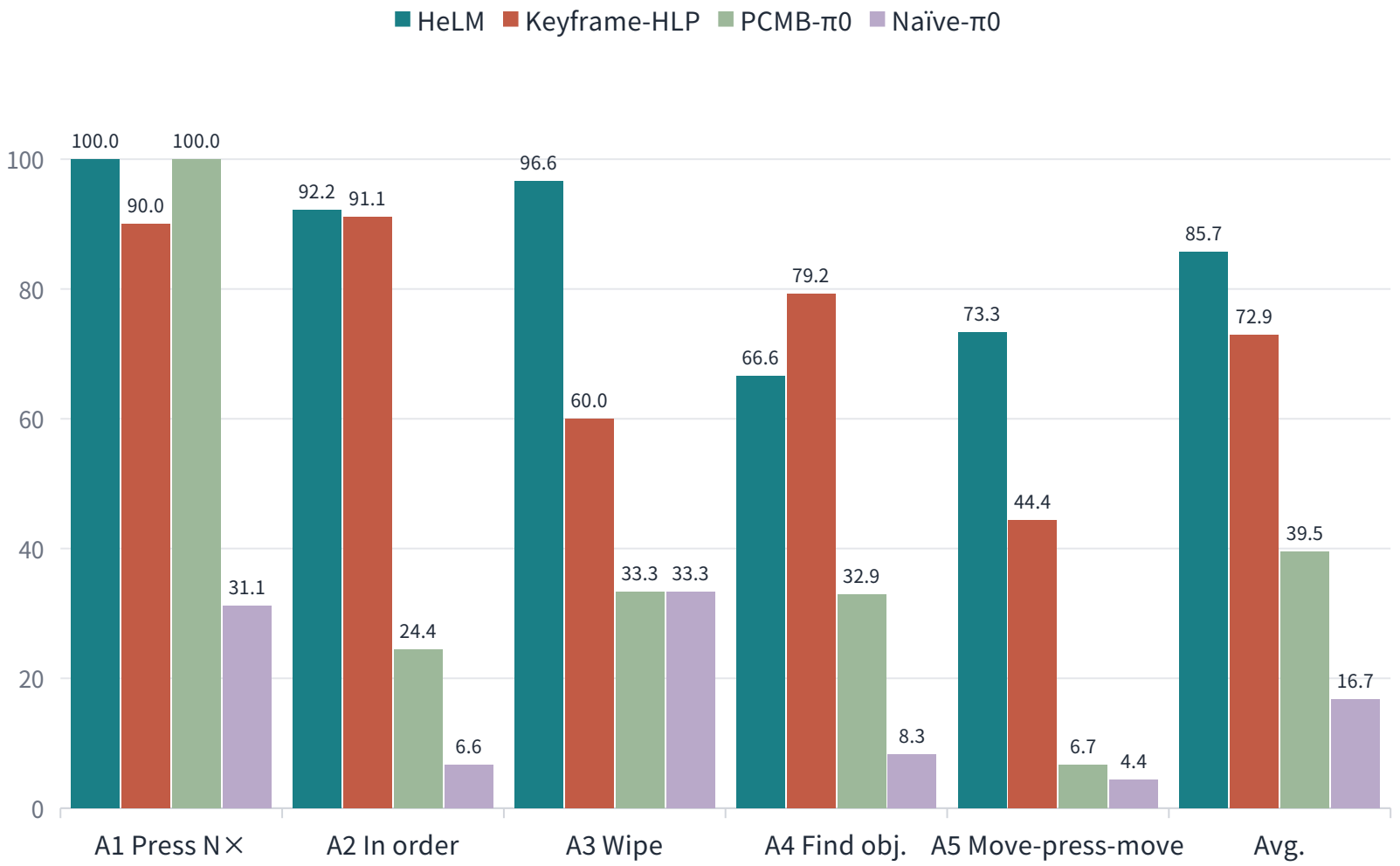
Intra-episode （A1–A5）

Working memory within one instruction: counting, ordering, wiped areas, searched drawers, multi-step stages

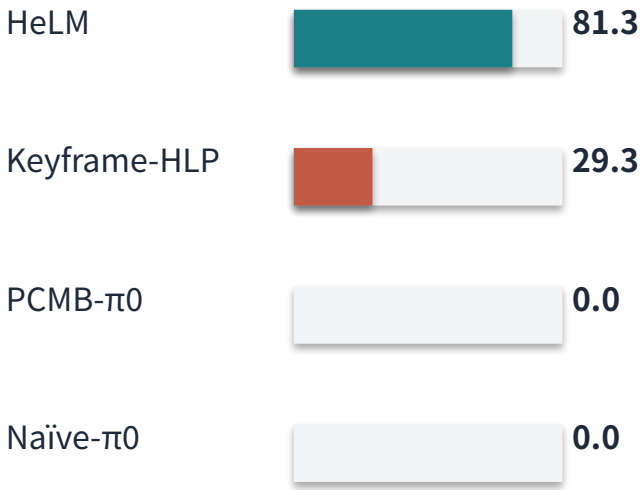
Inter-episode （E1–E5）

Later instructions depend on earlier episodes: cumulative counts, human-demo order, remaining areas, object locations, original positions

Intra-episode: HeLM averages 85.7%, with almost no strict-success drop



Strict SR (order + stop)

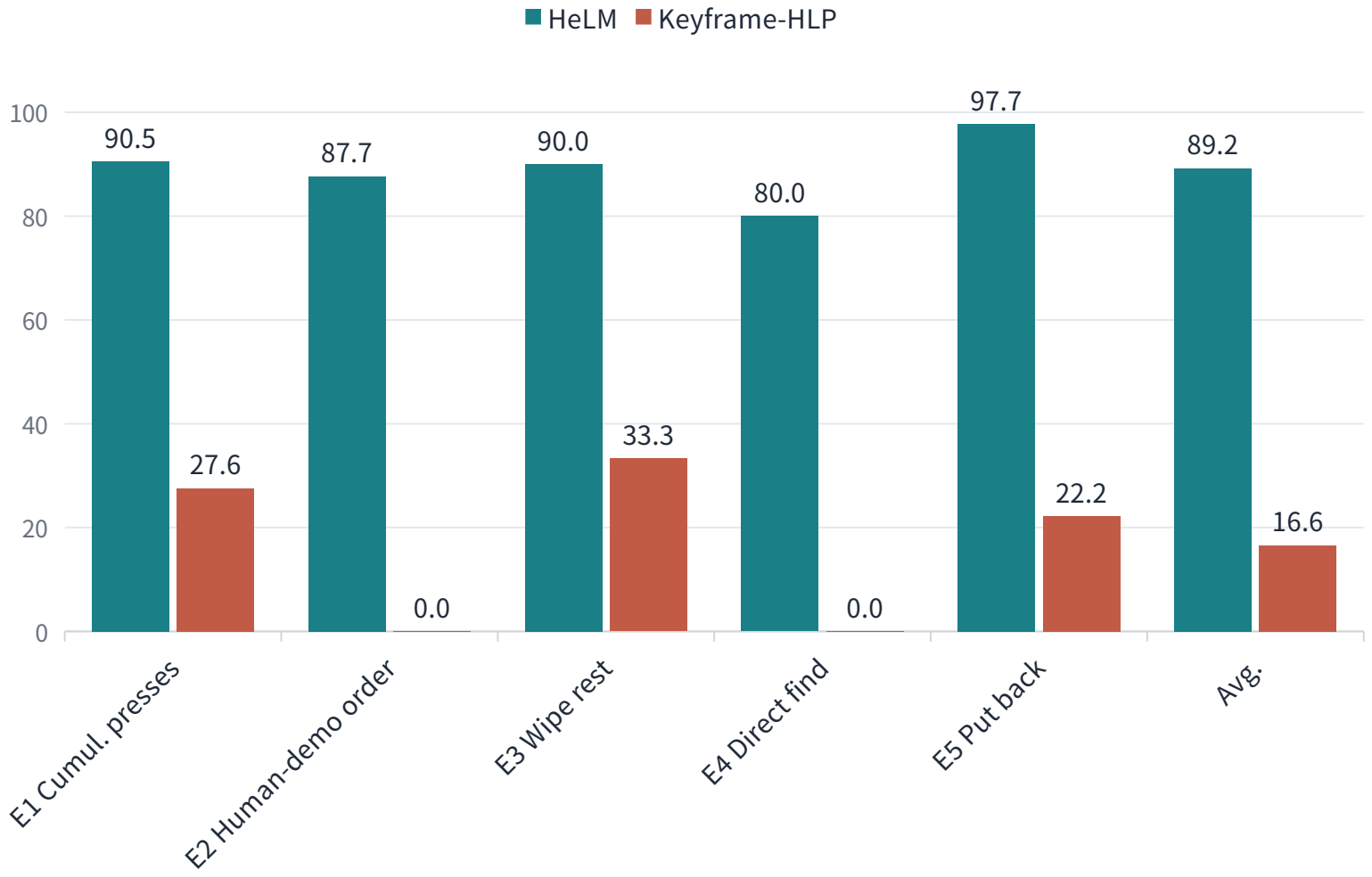


Key: HeLM drops only ~4 pp from subtask 85.7 to strict 81.3; baselines fall sharply due to ordering and termination errors.

On A4, Keyframe-HLP scores higher on subtasks (79.2*) but fails to terminate.

Source: Fig.5 (subtask SR), Tab.8 (strict SR)

Inter-episode: HeLM 89.2% vs keyframe baseline 16.6%



+72.6 pp

Average lead

89.2 vs 16.6 (subtask success rate)

Why keyframes fail

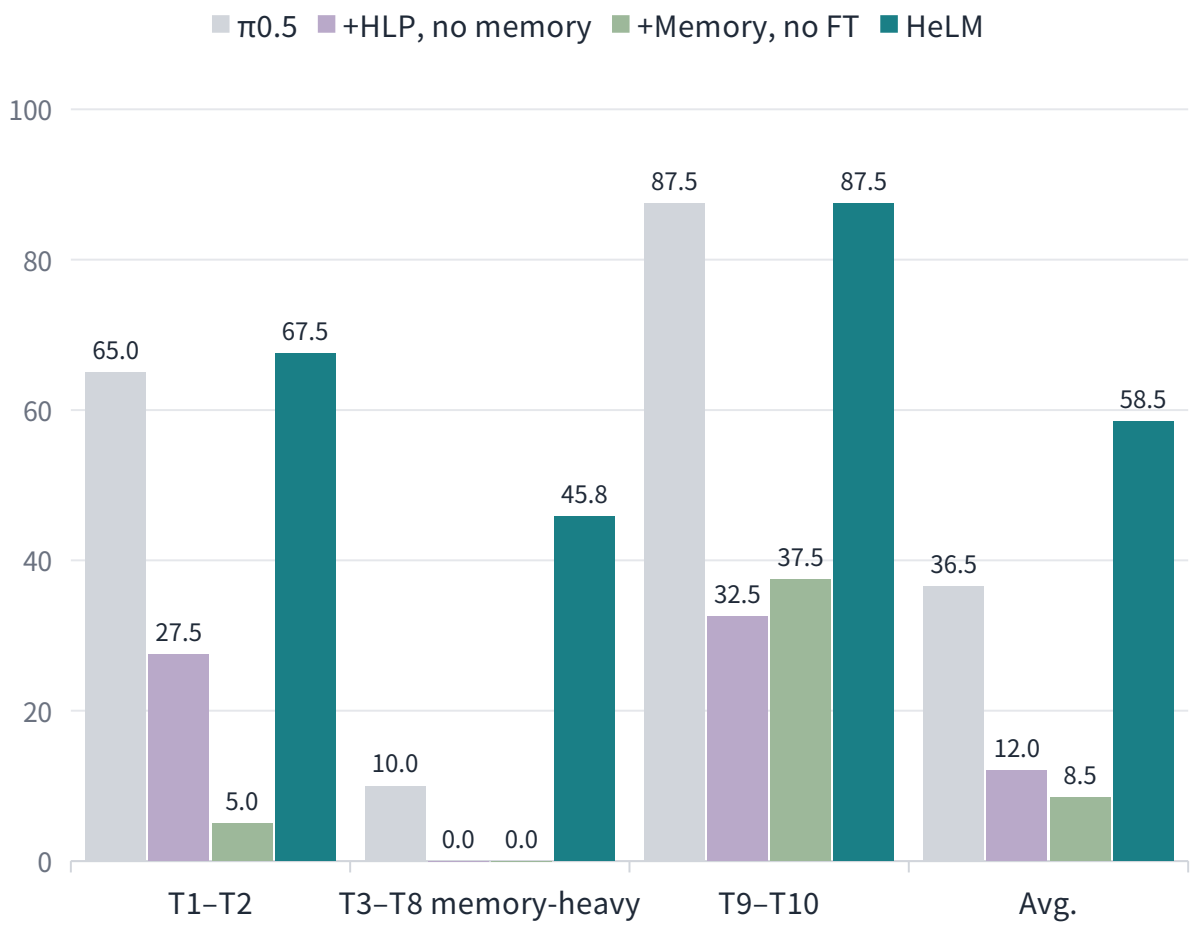
Images pile up across instructions, hard to read as coherent progress; E1 also shows termination failures (*)

Note

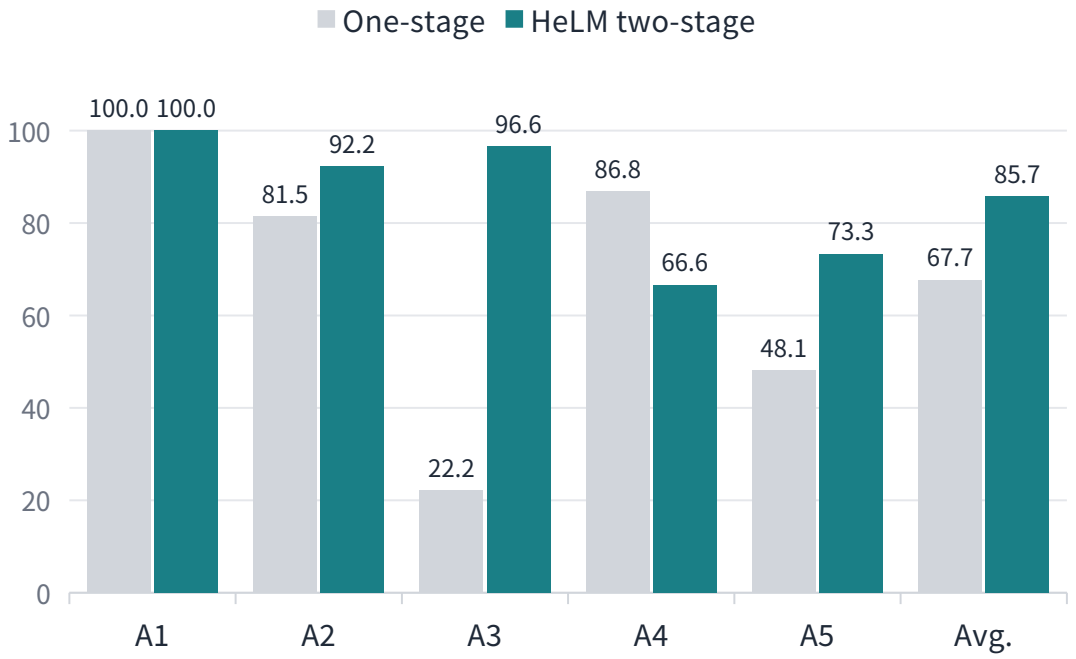
π0-family baselines reset at episode boundaries; naive context concatenation fails, so they are excluded

Memory, HLP fine-tuning and two-stage reasoning are all essential

LIBERO-Mem (LLP = π0.5) task success %



Real robot: two- vs one-stage subtask SR %



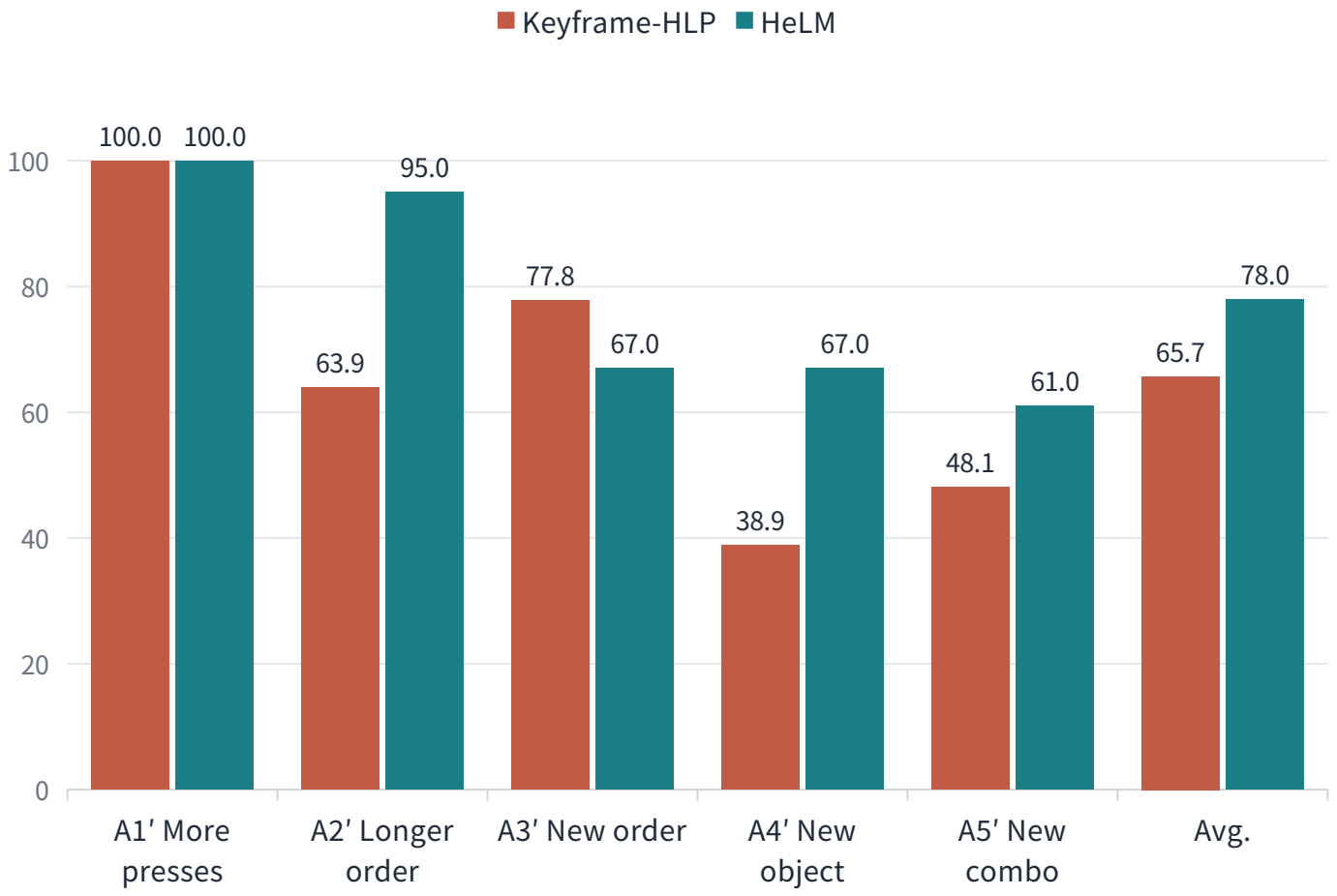
T3–T8: both ablated variants score 0%; HeLM 45.8%.

One-stage mainly fails by premature completion and wrong memory updates (A3 drops to 22.2); yet one-stage is higher on A4.

Source: Tab.2 (LIBERO-Mem), Tab.3 (real robot)

Unseen combos 78% w/o FT; small models: less memory, lower SR

Compositional generalization (A1′–A5′, no extra HLP FT) subtask SR %



Small-model trade-offs (Tab.10 / App. F.2)

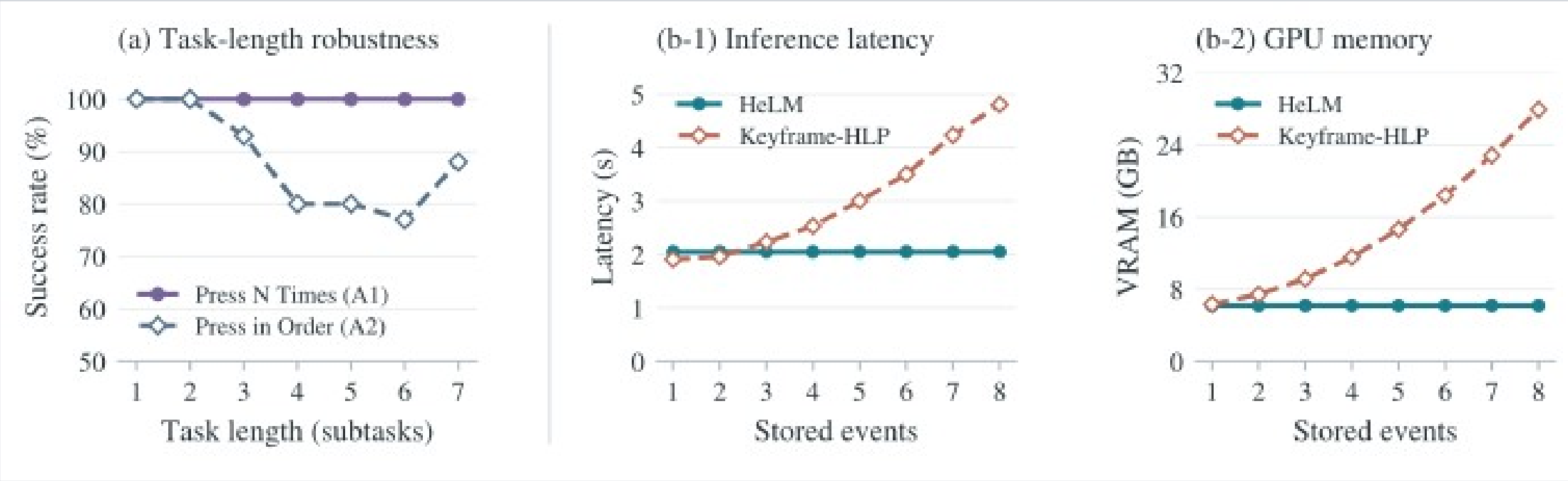
	HeLM (base)	HeLM (Smol)
HLP / LLP	Qwen2.5-VL 7B / π0 3.5B	SmolVLM 500M / SmolVLA 450M
Avg. success	**85.7%**	**58.7%**
Peak memory	**13.18 GB**	**2.3 GB**
E2E latency	**1.9 s**	**0.7 s**

Note: on A3′, HeLM (67.0) trails Keyframe-HLP (77.8); Keyframe-HLP fails to terminate on A1′/A4′/A5′.

Small-model failures center on UPDATE: invalid YAML and changing multiple state variables at once.

Source: Tab.4, Tab.10, Appendix F.2 •

Text memory cost is constant: more events, bigger edge over keyframes



100%

A1 Press N times (N = 1…7)

⩾ 77%

A2 Order length 1…7

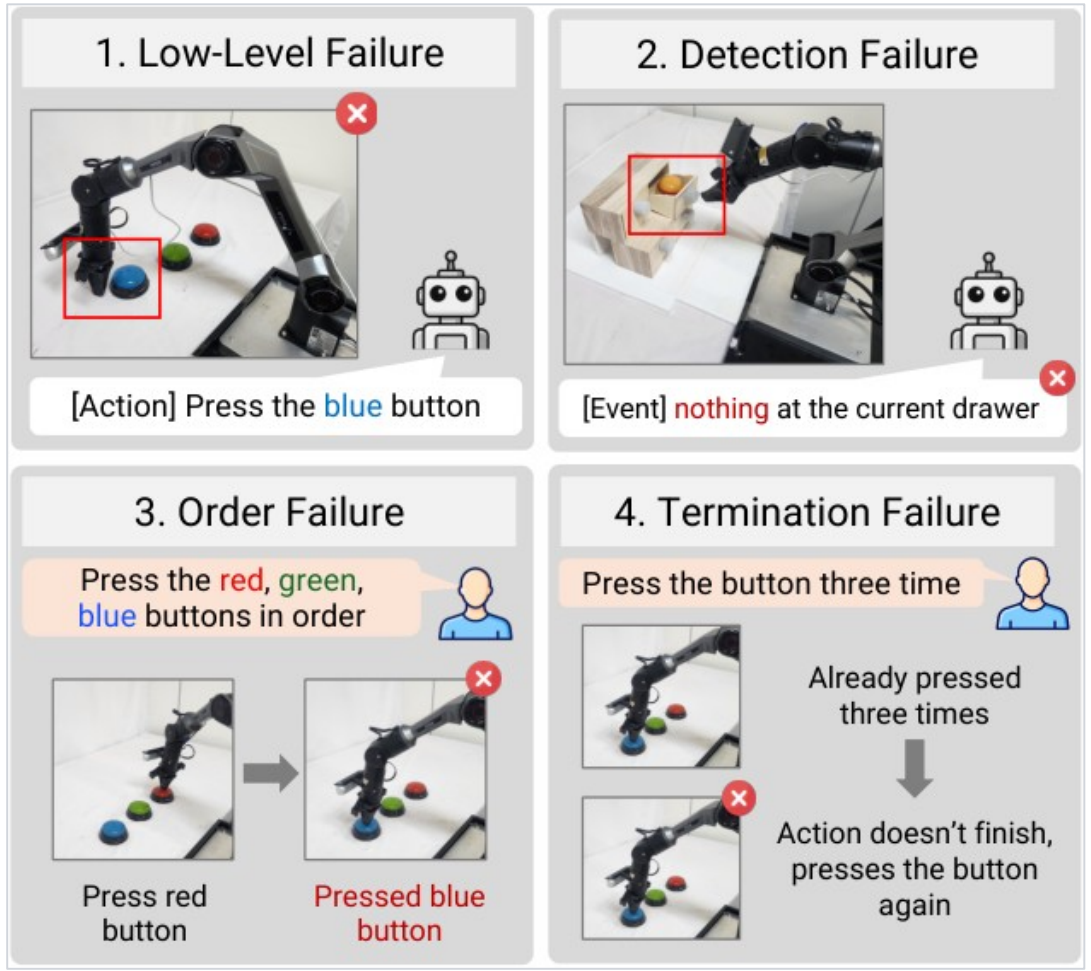
< 6 GB vs 28 GB

Peak memory at 8 events

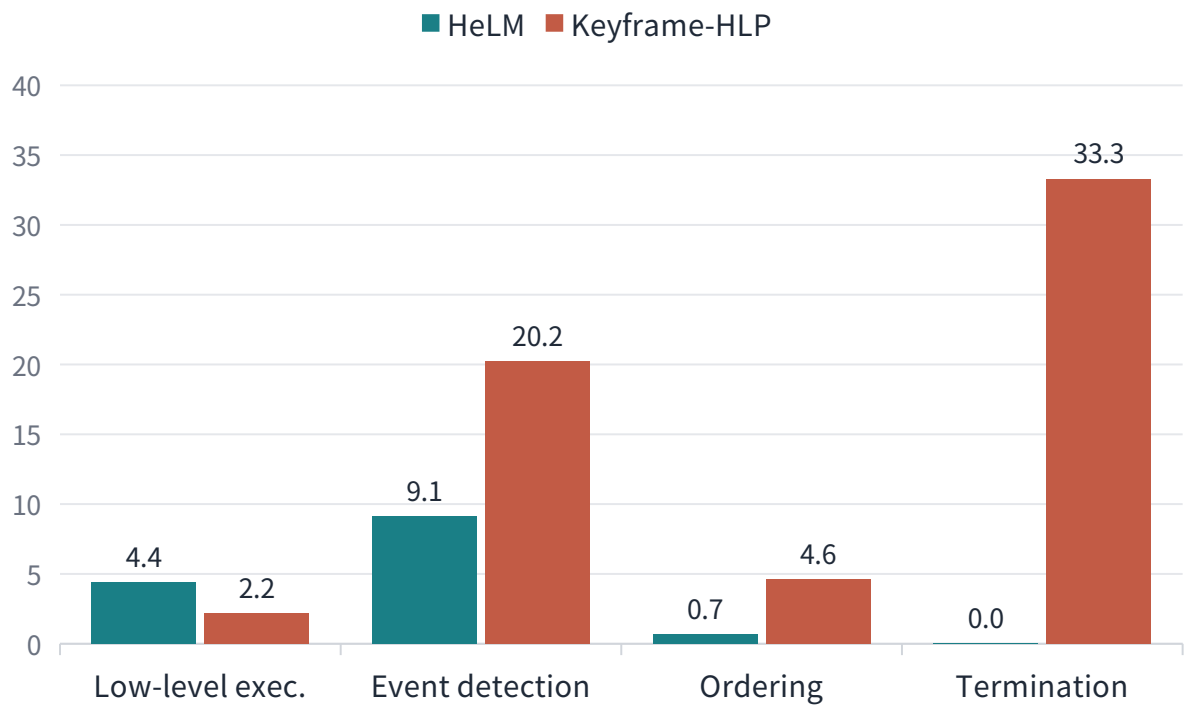
2.5×

Long-horizon speedup

HeLM: perception/execution errors, not memory; baselines: stop/order



Failure types as % of trials (A1–A5 avg., %)



HeLM failure breakdown: detection 64.1% • low-level 31.2% • ordering 4.7% • termination 0%

Detection failures are mostly false positives (A4/E4: drawer wrongly judged open); once written to memory, they cannot be self-corrected.

Source: Tab.7, Fig.8, Sec.5.3 •

Interpretable, cheap; limits: manual events, eval. scale

Strengths

Interpretable: memory is readable text; failures traceable to detection / update / execution

Well-grounded design: detect-update split, memory and fine-tuning all backed by ablations

Cheap data: short clips + TaskSpec stitching; no long-horizon demos needed

Clean comparison: Keyframe-HLP shares the HLP and event supervision, differing only in memory modality

New axis: proposes inter-episode memory tasks (E1–E5)

Limitations / Caveats

Relies on TaskSpec: events and memory transitions are hand-defined; "what to remember" remains unsolved

Discrete events only: no continuous states such as liquid levels or geometric trajectories

Uncorrectable misdetections: false positives propagate once in memory; synchronous calls may miss transient intra-chunk events

Baseline strength: PCMB is a scaled-down port; π0 baselines lack a termination mechanism

Eval. scale: 15–45 trials per task; no edge on A4, A3′, etc.; benefit of the dual-memory split unverified

Conclusions & Takeaways

Memory representation is a key design axis for robot policies

01 Language is a natural memory for VLM policies

Compact, readable, constant cost: <6 GB at 8 events vs 28 GB for keyframes

02 Event-triggered updates yield reliable ordering and termination

Intra strict success 81.3%; inter-episode tasks 89.2%

03 The next bottleneck is perception and execution

Event misdetection and contact-sensitive actions dominate failures; discovering "what to remember" remains open